

文章笔记

50 个 Claude Code 实用技巧与最佳实践

基于公开课程资料整理

2026 年 3 月 19 日

文章作者: Vishwas (CodevolutionWeb)

发布日期: 2026 年 3 月 19 日

阅读时长: 约 15 分钟阅读

文章链接: <https://x.com/CodevolutionWeb/status/2034683638382506063>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言	3
2 快速上手与基础操作	3
2.1 设置 cc 别名 (Tip #1)	3
2.2 用! 前缀直接执行 Bash 命令 (Tip #2)	3
2.3 Esc 中断与 Esc+Esc 回滚 (Tip #3)	3
2.4 常用快捷键汇总 (Tips #16, #17, #27)	4
2.5 命名与着色会话 (Tip #47)	4
2.6 自定义 Spinner 动词 (Tip #50)	4
2.7 本章小结	4
3 Prompt 工程与交互策略	4
3.1 用 ultrathink 触发深度推理 (Tip #7)	4
3.2 用 Plan Mode 规划复杂任务 (Tip #11)	5
3.3 直接贴原始数据, 不要自己解释 Bug (Tip #13)	5
3.4 用 @ 指定文件路径 (Tip #25)	5
3.5 用模糊 Prompt 探索不熟悉的代码 (Tip #26)	5
3.6 让 Claude 反过来采访你 (Tip #44)	5
3.7 用语音输入写更丰富的 Prompt (Tip #23)	5
3.8 本章小结	6
4 会话管理与上下文控制	6
4.1 不同任务之间用 /clear 清空 (Tips #12, #24)	6
4.2 扩展上下文窗口到 1M Tokens (Tip #10)	6
4.3 引导 Compaction 保留关键信息 (Tip #21)	6
4.4 用 /btw 快速提问 (Tip #14)	6
4.5 用 /branch 尝试不同方案 (Tip #43)	6
4.6 用 /loop 做周期性检查 (Tip #22)	7
4.7 本章小结	7
5 CLAUDE.md 与项目配置	7
5.1 用 /init 生成然后精简 (Tip #28)	7
5.2 每一行都用 Litmus Test 检验 (Tip #29)	7
5.3 让 Claude 自己更新 CLAUDE.md (Tip #30)	7
5.4 用 .claude/rules/ 做条件加载 (Tip #31)	7
5.5 用 @imports 按需引入 (Tip #32)	8
5.6 本章小结	8
6 权限、安全与 Hooks	8
6.1 CLAUDE.md 是建议, Hooks 是规则 (Tip #38)	8
6.2 用 /permissions 白名单化安全命令 (Tip #33)	8
6.3 用 /sandbox 隔离运行 (Tip #34)	8
6.4 PostToolUse Hook: 自动格式化 (Tip #39)	9

6.5	PreToolUse Hook: 拦截危险命令 (Tip #40)	9
6.6	Notification Hook: 跨 Compaction 保留上下文 (Tip #41)	9
6.7	Stop Hook: 完成时播放提示音 (Tip #48)	9
6.8	始终手动 Review 关键逻辑 (Tip #42)	10
6.9	本章小结	10
7	自动反馈与质量保证	10
7.1	给 Claude 一个反馈循环 (Tip #4)	10
7.2	安装语言级 Code Intelligence 插件 (Tip #5)	10
7.3	用 gh CLI 和其他命令行工具 (Tip #6)	10
7.4	对话式 PR Review (Tip #46)	11
7.5	本章小结	11
8	插件、MCP 与 Skills	11
8.1	Skills: 按需加载的知识 (Tip #8)	11
8.2	选择合适的 MCP Server (Tip #36)	11
8.3	设置输出风格 (Tip #37)	12
8.4	本章小结	12
9	并行工作与多 Agent 协作	12
9.1	用 -worktree 做并行隔离开发 (Tip #15)	12
9.2	用 Subagent 保持主上下文干净 (Tip #19)	12
9.3	自定义 Subagent (Tip #35)	12
9.4	Agent Teams: 多会话协调 (Tip #20)	13
9.5	一个 Claude 写代码, 另一个 Claude 审查 (Tip #45)	13
9.6	用 claude -p 做批量操作 (Tip #49)	13
9.7	远程控制 (Tip #9)	13
9.8	添加 Status Line (Tip #18)	13
9.9	本章小结	13
10	总结与延伸	13
10.1	拓展阅读	14

1 引言

Vishwas 是 Claude Code 社区中最活跃的内容创作者之一。他在这篇文章中汇总了 50 条 Claude Code 使用技巧，来源包括 Anthropic 官方文档、Claude Code 核心开发者 Boris Cherny 的分享、社区经验以及他本人一年多的日常使用心得。

本文将这 50 条技巧按主题重新组织为 8 个板块，帮助读者系统性地掌握 Claude Code 从入门到进阶的完整工作流。

文章背景

Claude Code 是 Anthropic 推出的官方命令行 AI 编程助手，运行在终端中，能够直接读写文件、执行 shell 命令、与 Git 交互。它支持 Sonnet 4.6 和 Opus 4.6 模型，最大上下文窗口可达 1M tokens。

2 快速上手与基础操作

掌握以下基础操作可以显著提升日常使用效率。

2.1 设置 cc 别名 (Tip #1)

在 `~/.zshrc` (或 `~/.bashrc`) 中添加别名，用 `cc` 代替 `claude` 启动会话，同时跳过所有权限确认弹窗。

```
1 alias cc="claude --dangerously-skip-permissions"
```

Listing 1: cc 别名配置

谨慎使用 `--dangerously-skip-permissions`

该 flag 的命名故意设计得“吓人”。只有在你完全理解 Claude Code 能对你的代码库做什么之后，才应启用此选项。它会跳过所有操作确认，包括文件删除和 shell 命令执行。

2.2 用! 前缀直接执行 Bash 命令 (Tip #2)

在 Claude Code 交互界面中输入 `!git status` 或 `!npm test`，命令会立即执行，输出会进入上下文供 Claude 参考。比让 Claude 帮你跑命令更快。

2.3 Esc 中断与 Esc+Esc 回滚 (Tip #3)

- **Esc**: 立即中断 Claude 的当前操作，不丢失上下文，可以马上重新引导。
- **Esc+Esc** (或 `/rewind`): 打开 checkpoint 列表，可恢复代码、对话或两者。四种恢复选项：代码 + 对话、仅对话、仅代码、从某个 checkpoint 开始摘要。

Checkpoint 的局限

Checkpoint 只追踪文件编辑操作。通过 Bash 命令造成的变更（数据库迁移、文件系统操作等）不会被 checkpoint 捕获，无法回滚。

恢复上次会话: `claude --continue` 恢复最近一次对话; `claude --resume` 打开会话选择器。

2.4 常用快捷键汇总 (Tips #16, #17, #27)

快捷键	功能
Ctrl+S	暂存当前正在编写的 prompt, 处理完其他事情后自动恢复
Ctrl+B	将长时间运行的 Bash 命令放入后台, Claude 继续工作
Ctrl+G	在文本编辑器中直接编辑 Claude 提出的计划
Shift+Tab	在 Normal / Auto-Accept / Plan 三种权限模式间切换
Esc	中断当前操作
Esc+Esc	打开 checkpoint 回滚菜单

表 1: Claude Code 常用快捷键

2.5 命名与着色会话 (Tip #47)

运行多个并行会话时, 用 `/rename auth-refactor` 给会话加标签, 用 `/color red` 设置提示栏颜色 (支持 red, blue, green, yellow, purple, orange, pink, cyan)。五秒钟的操作能避免你在错误的终端里输入命令。

2.6 自定义 Spinner 动词 (Tip #50)

Claude 思考时终端显示的 spinner 动词(如"Flibbertigibbet...")可以自定义。只需告诉 Claude: "Replace my spinner verbs with Harry Potter spells.", Claude 会自动生成列表。虽然是小事, 但能让等待变得有趣。

2.7 本章小结

基础操作的核心在于: 用别名简化启动、用 `!` 前缀快速执行命令、用 `Esc` 系列快捷键控制流程、用命名和颜色管理多会话。这些习惯一旦养成, 日常交互效率会有质的提升。

3 Prompt 工程与交互策略

如何与 Claude 对话直接决定输出质量。以下技巧帮助你写出更有效的 prompt。

3.1 用 ultrathink 触发深度推理 (Tip #7)

`ultrathink` 是一个关键词, 会将 `effort` 设为最高并触发 Opus 4.6 的自适应推理模式。Claude 会根据问题复杂度动态分配 thinking tokens。

何时使用 `ultrathink`

适用于: 架构决策、复杂 debug、多步推理。不适用于: 变量重命名等简单任务 (浪费 thinking tokens)。也可以用 `/effort` 命令永久设置推理强度。

3.2 用 Plan Mode 规划复杂任务 (Tip #11)

对于多文件变更、不熟悉的代码、架构决策，先进入 Plan Mode (Shift+Tab 切换)。虽然多花几分钟，但能防止 Claude 花 20 分钟自信地解决错误的问题。

对于一句话就能描述的小改动，直接执行即可，跳过规划。

3.3 直接贴原始数据，不要自己解释 Bug (Tip #13)

常见误区：用文字描述 Bug

用文字描述 bug 往往会丢失 Claude 定位根因所需的细节。直接粘贴 error log、CI 输出或 Slack 对话，然后说“fix”。Claude 擅长从原始日志中追踪问题。

也可以直接从终端 pipe 输出：

```
1 npm test 2>&1 | claude "fix the failing tests"
```

Listing 2: 管道传输错误信息

3.4 用 @ 指定文件路径 (Tip #25)

使用 @src/auth/middleware.ts 直接引用文件。Claude 虽然能自己搜索代码库，但每一步搜索都消耗 tokens 和上下文。直接指定文件跳过整个搜索过程。

3.5 用模糊 Prompt 探索不熟悉的代码 (Tip #26)

“What would you improve in this file?” 是一个很好的探索型 prompt。不是每个 prompt 都需要精确。当你想要新视角审视现有代码时，模糊问题给 Claude 发挥空间，能发现你自己不会想到去问的问题。

3.6 让 Claude 反过来采访你 (Tip #44)

当你无法完整描述需求时，让 Claude 来提问：

Interview 模式

告诉 Claude: “I want to build [feature]. Interview me to understand the full requirements before writing any code. Ask me one question at a time.” Claude 会逐步引导你补全需求细节。完成后，开一个新会话用干净上下文执行。

3.7 用语音输入写更丰富的 Prompt (Tip #23)

运行 /voice 启用按键说话功能，按住空格键口述。语音 prompt 自然比打字包含更多上下文，因为口述时你会不自觉地解释背景、提到约束条件。需要 Claude.ai 账号（非 API key）。可以在 ~/.claude/keybindings.json 中自定义按键。

3.8 本章小结

Prompt 工程的核心原则：复杂任务用 `ultrathink` 和 Plan Mode；贴原始数据而非文字描述；用 `@` 精确指向文件；在需求不明确时让 Claude 反向采访你。匹配推理强度与任务复杂度是控制成本的关键。

4 会话管理与上下文控制

Claude Code 的输出质量与上下文状态密切相关。管理好会话是长时间使用的关键。

4.1 不同任务之间用 `/clear` 清空 (Tips #12, #24)

最重要的会话管理原则

一个干净会话配合精准 prompt，胜过一个混乱的三小时长会话。不同任务？先 `/clear`。当你在同一个问题上纠正 Claude 超过两次，也应该 `/clear` 重新开始——上下文中堆积的失败尝试会“污染”后续输出。

4.2 扩展上下文窗口到 1M Tokens (Tip #10)

Sonnet 4.6 和 Opus 4.6 都支持 1M token 上下文窗口。在 Max/Team/Enterprise 计划中，Opus 自动升级到 1M。也可以在会话中用 `/model opus[1m]` 切换。

相关环境变量：

- `CLAUDE_CODE_AUTO_COMPACT_WINDOW`：控制自动 compaction 触发时机
- `CLAUDE_AUTOCOMPACT_PCT_OVERRIDE`：设置 compaction 百分比阈值

4.3 引导 Compaction 保留关键信息 (Tip #21)

当上下文自动压缩时，用命令引导 Claude 保留什么：

```
1 /compact focus on the API changes and the list of modified files
```

Listing 3: 引导 compaction 示例

也可以在 `CLAUDE.md` 中添加常驻指令：“When compacting, preserve the full list of modified files and current test status.”

4.4 用 `/btw` 快速提问 (Tip #14)

`/btw` 弹出一个覆盖层用于快速提问，不会进入对话历史。适合问“为什么选了这个方案？”或“另一个选项的权衡是什么？”主上下文保持干净。

4.5 用 `/branch` 尝试不同方案 (Tip #43)

`/branch` (或 `/fork`) 在当前节点创建对话副本。在分支中尝试激进重构，如果成功就保留，失败了原始对话不受影响。与 `Esc+Esc` 回滚不同，两条路径同时存活。

4.6 用 /loop 做周期性检查 (Tip #22)

```
1 /loop 5m check if the deploy succeeded and report back
2 /loop 20m /review-pr 1234
```

Listing 4: /loop 示例

支持 s, m, h, d 时间单位，默认 10 分钟。任务绑定到会话，3 天后过期，不会无限运行。适合监控部署、CI 流水线等场景。

4.7 本章小结

会话管理的关键：不同任务间 /clear、修正两次仍失败就重新开始、引导 compaction 保留关键信息、用 /btw 和 /branch 保持上下文干净。理解上下文窗口和 compaction 机制是高效使用 Claude Code 的基础。

5 CLAUDE.md 与项目配置

CLAUDE.md 是项目级的持久化配置，相当于给 Claude Code 的”项目说明书”。

5.1 用 /init 生成然后精简 (Tip #28)

CLAUDE.md 是项目根目录下的 Markdown 文件，给 Claude 提供持久指令：构建命令、编码规范、架构决策、仓库约定。每次会话开始时 Claude 都会读取它。

运行 /init 自动生成初始版本，但输出通常很臃肿。精简原则：如果你说不出某一行为什么存在，就删掉它。

5.2 每一行都用 Litmus Test 检验 (Tip #29)

CLAUDE.md 的黄金法则

对 CLAUDE.md 的每一行都问：没有这行，Claude 会犯错吗？如果 Claude 本来就做得对，这行就是噪声。大约有 150–200 条指令的”预算”，超过后遵从度下降，而系统 prompt 已经占用了约 50 条。

5.3 让 Claude 自己更新 CLAUDE.md (Tip #30)

当 Claude 犯错时，说”update the CLAUDE.md file so this doesn’t happen again”。Claude 会写入自己的规则，下次自动遵守。随着时间推移，CLAUDE.md 变成一个由真实错误驱动的活文档。

为防止文件无限膨胀，用 @imports 引用外部文件（如 @docs/solutions.md），CLAUDE.md 本身保持精简。

5.4 用 .claude/rules/ 做条件加载 (Tip #31)

在 .claude/rules/ 目录下放置 Markdown 文件，按主题组织规则。通过 frontmatter 中的 paths 字段控制加载时机：


```

1 ---
2 paths:
3   - "**/*.ts"
4   - "**/*.tsx"
5 ---
6 # TypeScript conventions
7 - Use strict mode
8 - Prefer interfaces over types

```

Listing 5: 条件加载规则示例

TypeScript 规则只在 Claude 处理 `.ts` 文件时加载，Go 规则只在处理 `.go` 文件时加载。

5.5 用 @imports 按需引入 (Tip #32)

用 `@docs/git-instructions.md`、`@README.md`、`@package.json` 甚至 `@~/.claude/my-project-instructions` 引用外部文档。Claude 在需要时才读取，不会每次会话都加载。

5.6 本章小结

CLAUDE.md 的核心理念：精简、针对性、活文档。用 `/init` 起步后立刻精简；每行都做 litmus test；让 Claude 从错误中自学；用 `.claude/rules/` 和 `@imports` 分散配置，保持主文件简洁。

6 权限、安全与 Hooks

Hooks 是 Claude Code 中确定性最高的控制机制，适合处理“必须每次都执行”的规则。

6.1 CLAUDE.md 是建议，Hooks 是规则 (Tip #38)

CLAUDE.md vs Hooks 的本质区别

CLAUDE.md 是建议性的，Claude 大约 80% 的时间会遵守。Hooks 是确定性的，100% 执行。如果某件事必须无条件发生（格式化、lint、安全检查），用 Hook；如果是 Claude 应该考虑的指导方针，用 CLAUDE.md。

6.2 用 /permissions 白名单化安全命令 (Tip #33)

不要再对 `npm run lint` 点一百次“approve”了。`/permissions` 让你将信任的命令加入白名单，不在列表中的命令仍然会弹出确认。

6.3 用 /sandbox 隔离运行 (Tip #34)

`/sandbox` 启用操作系统级隔离：文件写入限制在项目目录，网络请求限制在你批准的域名。macOS 使用 Seatbelt，Linux 使用 bubblewrap。

无人值守场景

对于过夜迁移、实验性重构等无人值守任务，推荐在 Docker 容器中运行 Claude。容器提供完全隔离和轻松回滚。

6.4 PostToolUse Hook：自动格式化 (Tip #39)

每次 Claude 编辑文件后自动运行格式化工具：

```
1 # PostToolUse hook
2 npx prettier --write "$CLAUDE_FILE_PATH" || true
```

Listing 6: PostToolUse Hook 配置 (.claude/settings.json)

`|| true` 防止格式化失败阻塞 Claude。注意：如果编辑器同时打开了相同文件，建议关闭 `format-on-save`，因为编辑器保存可能使 `prompt cache` 失效。

6.5 PreToolUse Hook：拦截危险命令 (Tip #40)

用 PreToolUse Hook 拦截 `rm -rf`、`DROP TABLE`、`TRUNCATE` 等危险命令。Hook 在执行前触发，危险命令不会被实际执行。

可以通过 `/hooks` 交互式设置，或直接告诉 Claude：“Add a PreToolUse hook that blocks `rm -rf`, `drop table`, and `truncate` commands.”

6.6 Notification Hook：跨 Compaction 保留上下文 (Tip #41)

长会话中 `compaction` 可能导致 Claude 丢失当前工作的上下文。设置一个 `Notification hook`，在 `compaction` 触发时自动重新注入关键信息：

- 当前任务描述
- 已修改的文件列表
- 硬性约束条件（如“不要修改 `migration` 文件”）

6.7 Stop Hook：完成时播放提示音 (Tip #48)

添加 `Stop hook`，Claude 完成响应时播放系统提示音。启动任务后切换到其他工作，听到提示音再回来查看。

6.8 始终手动 Review 关键逻辑 (Tip #42)

人工审核不可省略的场景

无论 Claude 生成的代码看起来多好，以下领域必须人工审核：

- 认证/授权流程 (Auth flows)
- 支付逻辑 (Payment logic)
- 数据变更/删除操作 (Data mutations)
- 破坏性数据库操作 (Destructive DB operations)

错误的 auth scope、配置错误的支付 webhook、或静默删除列的 migration 可能造成用户流失、资金损失或信任危机。

6.9 本章小结

安全控制的层次结构：/permissions 管理命令白名单 → /sandbox 提供系统级隔离 → Pre-ToolUse Hook 拦截危险操作 → PostToolUse Hook 自动格式化 → Notification Hook 保留上下文。CLAUDE.md 管建议，Hook 管规则，关键逻辑必须人工审核。

7 自动反馈与质量保证

让 Claude 自己验证自己的输出是提升质量最有效的方式。

7.1 给 Claude 一个反馈循环 (Tip #4)

2-3x 质量提升

在 prompt 中包含测试命令、linter 检查或预期输出。Claude 运行测试、看到失败、自动修复，无需你介入。Claude Code 核心开发者 Boris Cherny 表示，仅此一项就能带来 2-3 倍的质量提升。

对于 UI 变更，配置 Playwright MCP server，让 Claude 能打开浏览器、与页面交互、验证 UI 是否正常工作。

7.2 安装语言级 Code Intelligence 插件 (Tip #5)

LSP 插件让 Claude 在每次文件编辑后自动获得诊断信息：类型错误、未使用的 import、缺失的返回类型。Claude 在你注意到之前就能看到并修复问题。

这是影响最大的单个插件

运行 /plugin 进入 Discover 标签浏览完整列表。支持 TypeScript、Python、Rust、Go、C#、Java、Kotlin、Swift、PHP 等语言。需要系统上安装对应的 Language Server。

7.3 用 gh CLI 和其他命令行工具 (Tip #6)

gh CLI 处理 PR、Issue、评论，不需要额外的 MCP server。CLI 工具比 MCP server 更节省上下文，因为不需要加载 tool schema。

对于 Claude 不认识的工具，告诉它：“Use `sentry-cli --help` to learn about it, then use it to find the most recent error in production.” Claude 会读 help 输出、理解语法、运行命令。

7.4 对话式 PR Review (Tip #46)

不要让 Claude 做“一次性” PR review。在会话中与 Claude 对话式审查：

- “Walk me through the riskiest change in this PR.”
- “What would break if this runs concurrently?”
- “Is the error handling consistent with the rest of the codebase?”

对话式审查能发现更多问题，因为你可以深入到重要区域。一次性审查往往只标记风格问题，容易遗漏架构问题。

7.5 本章小结

质量保证的核心策略：在 prompt 中嵌入反馈循环（测试、lint）、安装 LSP 插件获取实时诊断、用 CLI 工具代替 MCP server 节省上下文、用对话式方法做 PR review。

8 插件、MCP 与 Skills

Claude Code 的扩展能力来自三个层面：Plugins、MCP Servers 和 Skills。

8.1 Skills：按需加载的知识 (Tip #8)

Skills 是 Markdown 文件，按需扩展 Claude 的知识。与每次会话都加载的 CLAUDE.md 不同，Skills 只在与当前任务相关时才加载。

在 `.claude/skills/` 目录创建，或通过 `/plugin` 安装自带 Skills 的插件。适合存放：API 约定、部署流程、编码模式等偶尔需要的专业领域知识。

8.2 选择合适的 MCP Server (Tip #36)

推荐的 MCP Server 起步组合：

- **Playwright**：浏览器测试和 UI 验证
- **PostgreSQL/MySQL**：直接查询数据库 schema
- **Slack**：读取 bug 报告和对话上下文
- **Figma**：设计稿转代码

Claude Code 支持动态工具加载（dynamic tool loading），server 定义只在 Claude 需要时才加载。

8.3 设置输出风格 (Tip #37)

运行 `/config` 选择输出风格:

- **Explanatory**: 详细、步骤式
- **Concise**: 简洁、行动导向
- **Technical**: 精确、专业术语

也可以在 `~/.claude/output-styles/` 下创建自定义输出风格文件。

8.4 本章小结

扩展能力选择策略: 频繁需要的指令放 `CLAUDE.md`, 偶尔需要的领域知识用 `Skills`, 外部服务集成用 `MCP Server`, 能用 `CLI` 的场景优先 `CLI`。输出风格按团队偏好设置。

9 并行工作与多 Agent 协作

Claude Code 支持多种并行工作模式, 从单机 `worktree` 到多 Agent 团队协作。

9.1 用 `--worktree` 做并行隔离开发 (Tip #15)

`claude --worktree feature-auth` 创建一个带新分支的隔离工作副本。Claude Code 团队称这是最大的生产力解锁之一。

Worktree 最佳实践

通常同时运行 2-3 个 `worktree`, 每个有独立会话、独立分支、独立文件系统状态。本地 `worktree` 的上限是你的机器资源 (CPU、内存)。如需更多并行度, 可以考虑云容器方案 (如 `Builder.io`)。

9.2 用 Subagent 保持主上下文干净 (Tip #19)

”Use subagents to figure out how the payment flow handles failed transactions.” 这会生成一个独立的 Claude 实例, 有自己的上下文窗口。它读取文件、推理后返回简洁摘要。

内置类型:

- **Explore** (Haiku): 快速文件搜索
- **Plan** (只读分析): 代码库推理

深度调查可能消耗一半上下文窗口, Subagent 将这些成本隔离在主会话之外。

9.3 自定义 Subagent (Tip #35)

与临时使用不同, 自定义 Subagent 保存在 `.claude/agents/`, 预配置好模型和工具权限。例如: 用 `Opus` 和只读工具的 `security-reviewer agent`, 或用 `Haiku` 追求速度的 `quick-search agent`。

用 `/agents` 浏览和创建。可以设置 `isolation: worktree` 让 agent 拥有独立文件系统。

9.4 Agent Teams: 多会话协调 (Tip #20)

实验性功能

需要先启用 `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` 环境变量。Team lead 分配任务给 teammates, 每个 teammate 有独立上下文窗口和共享任务列表, 可以直接互相通信。

建议从 3-5 个 teammates、每人 5-6 个任务开始。避免让两个 teammate 编辑同一文件 (会导致覆盖)。先从研究和审查任务开始练手。

9.5 一个 Claude 写代码, 另一个 Claude 审查 (Tip #45)

第一个 Claude 实现功能, 第二个 Claude 从全新上下文出发像 Staff Engineer 一样审查。审查者不知道实现过程中的偷懒之处, 会挑战每一个取巧。

同样的思路适用于 TDD: Session A 写测试, Session B 写代码通过测试。

9.6 用 `claude -p` 做批量操作 (Tip #49)

```
1 for file in src/*.ts; do
2   claude -p "Update imports in $file" \
3     --allowedTools Write,Read &
4 done
5 wait
```

Listing 7: 批量操作示例

`--allowedTools` 限定每个文件允许的操作, `&` 并行执行。适合格式转换、批量更新 import、独立文件的重复迁移。

9.7 远程控制 (Tip #9)

运行 `claude remote-control` 启动会话, 然后从 `claude.ai/code` 或 iOS/Android 上的 Claude App 连接。会话在本地运行, 手机/浏览器只是窗口。可以发消息、审批工具调用、随时监控进度。

9.8 添加 Status Line (Tip #18)

Status line 是每个 Claude 回合后运行的 shell 脚本, 在终端底部显示实时信息: 当前目录、Git 分支、上下文使用量 (按窗口占用率着色)。用 `/statusline` 快速设置。

9.9 本章小结

并行工作的层次: 单任务用 Subagent 隔离调查成本 → 多分支用 Worktree 并行开发 → 跨会话用 Agent Teams 协作 → 批量文件用 `claude -p` 扇出处理。远程控制让你可以移动端监控长任务。

10 总结与延伸

这 50 条技巧覆盖了 Claude Code 使用的完整生命周期:

1. 基础操作: 别名、快捷键、会话管理构成日常交互的骨架。

2. **Prompt 工程**: 匹配推理强度与任务复杂度, 贴原始数据而非解释, 让 Claude 采访你补全需求。
3. **上下文控制**: 频繁 `/clear`、引导 compaction、用 `/btw` 和 `/branch` 保持干净。
4. **项目配置**: CLAUDE.md 精简且活、`.claude/rules/` 条件加载、`@imports` 按需引入。
5. **安全与 Hooks**: CLAUDE.md 管建议, Hooks 管规则, 关键逻辑必须人工审核。
6. **质量保证**: 反馈循环 (测试 + lint)、LSP 插件、对话式 PR review。
7. **扩展能力**: Skills 按需加载、MCP Server 集成外部服务、CLI 优先。
8. **并行工作**: Worktree、Subagent、Agent Teams、`claude -p` 各有适用场景。

最核心的建议

正如原文作者所说: 你不需要全部 50 条。挑出上一次会话中最让你烦恼的那个问题, 找到对应的技巧, 明天就试用。一条真正落地的技巧, 胜过五十条收藏夹里的书签。

10.1 拓展阅读

- Anthropic 官方 Claude Code 文档: <https://docs.anthropic.com/en/docs/claude-code>
- Boris Cherny 关于 Claude Code 架构的分享
- Vishwas 的 Claude Code 系列文章: <https://x.com/CodevolutionWeb>
- Claude Code Hooks 官方指南
- MCP Server 生态目录